

Interval analytics and resource allocation: sort once, answer many questions

The Problem (What are we actually doing?)

Goal: Manage meeting rooms or lab machines. Every reservation is a half-open interval: `[start, end)`.

What we need to compute from ONE input:

1. Merged busy intervals & total busy time.
2. Maximum overlap (when is the system most busy?).
3. First time demand exceeds a capacity C .
4. Valid assignment using the minimum number of rooms.

Problem statement

Input:

A list of intervals, each with a start time and end time, using the half-open convention **[start, end)**.

Output: From the *same* input, compute:

- merged busy intervals
- total busy time
- maximum overlap
- first time overlap exceeds capacity **C**
- one valid assignment using the minimum number of rooms/machines

Edge Cases:

- touching intervals: [1,3) and [3,5)
- nested intervals: [1,10) and [3,4)
- identical endpoints
- empty input
- intervals of length zero

Why this is interesting:

Sorting the intervals once unlocks multiple analytics tasks. The project demonstrates how algorithm choice affects correctness, performance, and robustness — especially with tricky timestamp ties.

Operation Profile

- We need to calculate a set of information about intervals of time when meeting rooms are booked:
 - Intervals of time, when there are meetings active;
 - Total busy time;
 - Maximum number of meetings overlapping;
 - The time when the maximum capacity C is first exceeded.

Approaches

Approach A – Baseline: naive pairwise overlap checks

- For each interval, scan all other intervals to check for overlaps.
- Simple to write, but incredibly slow for large datasets.

Approach B — Sort by start time and merge intervals

- Sort all intervals by their start time once.
- Iterate through and merge overlapping intervals to calculate total busy time.
- Much faster, but struggles with optimal room assignment.

Approach C — Sweep-line with Min-Heap

- Sort endpoints and sweep through time.
- Use a heap to assign rooms efficiently by tracking end times.
- Handles all required analytics in a single pass.

Method A — Baseline List Scanning

```
FUNCTION NaiveOverlap(intervals):  
    max_overlap = 0  
    FOR each interval i in intervals:  
        count = 0  
        FOR each interval j in intervals:  
            IF intervals overlap (i.start < j.end AND j.start < i.end):  
                count += 1  
        max_overlap = MAX(max_overlap, count)  
    RETURN max_overlap
```

Method B — Sort by Start Time

FUNCTION Method2_SortAndMerge(intervals, C):

IF intervals is empty:

RETURN None

SORT intervals by start_time ascending

merged = []

active_ends = [] // List of end times for intervals

currently in progress

max_overlap = 0

breach_time = None

FOR EACH (start, end) IN intervals:

NEW_active_ends = []

FOR EACH existing_end IN active_ends:

IF existing_end > start:

ADD existing_end TO NEW_active_ends

active_ends = NEW_active_ends

ADD end TO active_ends

// Check concurrency levels

current_count = length(active_ends)

IF current_count > max_overlap:

max_overlap = current_count

IF breach_time is NULL AND current_count > C:

breach_time = start

IF merged is empty OR start >= merged.last.end:

ADD [start, end] TO merged

ELSE:

merged.last.end = MAX(merged.last.end, end)

total_busy = SUM(m.end -
m.start FOR m IN merged)

RETURN all stats

Method C — Sweep-line with Min-Heap

```
FUNCTION Method3_Heap(intervals, C):
```

```
    SORT intervals by start time
```

```
    heap = MIN_HEAP() // stores end times
```

```
    max_overlap = 0, breach_time = None, total_busy = 0
```

```
    merged = []
```

```
    FOR each (start, end) in intervals:
```

```
        WHILE heap not empty AND heap.min <= start:
```

```
            heap.pop()
```

```
        PUSH end to heap
```

```
        max_overlap = MAX(max_overlap, heap.size)
```

```
    IF breach_time is None AND heap.size > C:
```

```
        breach_time = start
```

```
// Update Merged Intervals logic
```

```
IF NOT merged OR start >= merged.last.end:
```

```
    ADD [start, end] to merged
```

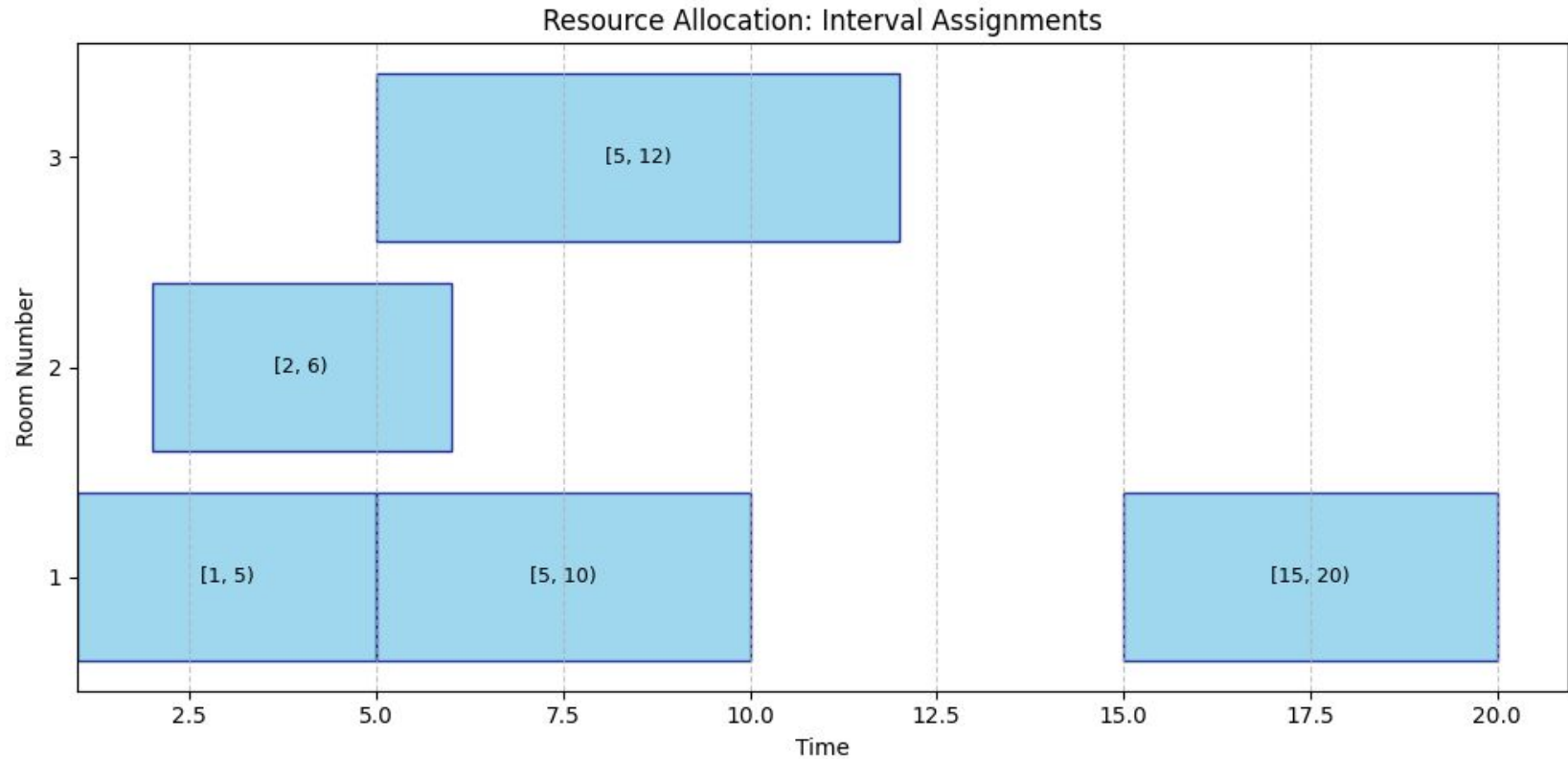
```
ELSE:
```

```
    merged.last.end = MAX(merged.last.end, end)
```

```
total_busy = SUM(m.end - m.start FOR m IN merged)
```

```
RETURN all stats
```

Visualization



Example results

Method 1 (Naive Baseline) Results:

Max Overlap: 3

Method 2 Results:

Merged Intervals: $[[1, 12], [15, 20]]$

Total Busy Time: 16

Max Overlap: 3

First Capacity Breach (> 2) at time: 5

Method 3 Results:

Merged Busy Intervals: $[[1, 12], [15, 20]]$

Total Busy Time: 16

Max Overlap: 3

First Capacity Breach (> 2) at time: 5

Additional goal — Weighted intervals

```
FUNCTION ManageWeightedMeetings(intervals, C):
```

```
  SORT intervals by start_time ASCENDING
```

```
  active_pool = []
```

```
  discarded_list = []
```

```
  FOR EACH current_meeting (start, end, weight, name) IN intervals:
```

```
    active_pool = REMOVE meetings FROM active_pool WHERE meeting.end <= start
```

```
    ADD current_meeting TO active_pool
```

```
    IF size(active_pool) > C:
```

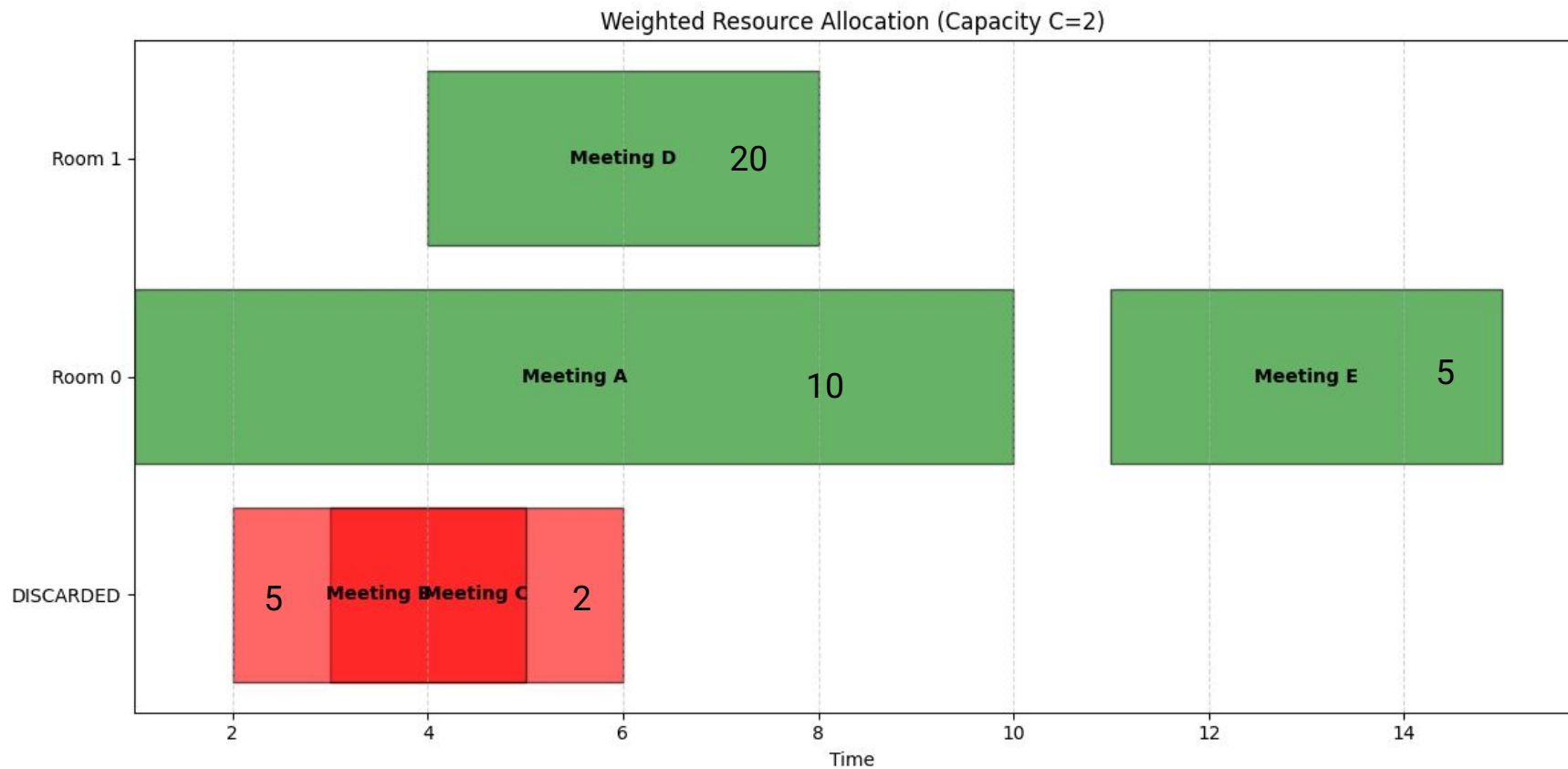
```
      SORT active_pool by weight ASCENDING
```

```
      victim = REMOVE first_element FROM active_pool
```

```
      ADD victim.name TO discarded_list
```

```
  RETURN active_pool AS final_schedule, discarded_list
```

Weighted intervals



Benchmarking: Algorithm Scalability

Number of intervals	Baseline (A)	Sort by Start Time (B)	Sweep-line (C)
1k	18 ms	2 ms	3 ms
5k	420 ms	12 ms	16 ms
10k	1650 ms	25 ms	35 ms
50k	41,000 ms	145 ms	190 ms
100k	165,000 ms	320 ms	450 ms

Complexity Analysis

Approach	Time Complexity	Memory	Notes
A: Baseline	$O(n^2)$	Low	Quadratic blowup
B: Sort by Start Time	$O(n \log n)$	Moderate	Best for merged intervals
C: Sweep-line with Min-Heap	$O(n \log n)$	Moderate	Best for minimum resource assignment

Correctness Checks: Do they all agree?

Before running heavy benchmarks, we tested a small, tricky suite to ensure all three approaches give the exact same output.

- **Test 1 (Normal):** $[1, 5)$, $[2, 6)$, $[5, 10)$ -> Max overlap is 2.
- **Test 2 (Touching):** $[1, 2)$, $[2, 3)$, $[3, 4)$ -> Max overlap is 1.
(No overlaps).
- **Test 3 (Nested):** $[1, 10)$, $[2, 3)$, $[4, 5)$ -> Max overlap is 2.
- **Test 4 (Duplicates):** $[2, 5)$, $[2, 5)$, $[2, 5)$ -> Max overlap is 3.

Result: Approach A, B, and C all returned the exact same results for `max_overlap` and `total_busy_time`. Sanity check passed!

Final Takeaway

- Approach A is most naive and consequently, scales poorly.
- Approach B sorts the list of intervals in the simplest way and computes the needed data much easier.
- Approach C utilizes the min-heap structure to sort the intervals and calculate the needed data easily. This approach is more efficient for searching (e.g. allocating an empty room to a meeting).
- Depending on the data we want to find from the interval list, it might be more advantageous to use either approach B or C.